

Data communications (Lab 8): Bipolar AMI Encoding

Objective: To generate and analyze a Bipolar Alternate Mark Inversion (AMI) encoded signal using MATLAB from a given binary data sequence.

Theory

Bipolar AMI (Alternate Mark Inversion) is a line coding technique used in digital communication where binary data is represented using three voltage levels:

- Binary 0 $\rightarrow$ 0 voltage level
- Binary 1 $\rightarrow$ alternating +1 and -1 voltage levels

The important rule is:

Every successive 1 changes polarity alternately:

+1,-1,+1,-1,...

This alternating polarity removes the DC component and improves synchronization.

Example: For binary sequence: 1 0 1 1 0 1 1

AMI output becomes: +1 0 -1 +1 0 -1

This is why it is called Alternate Mark Inversion:

- Mark (1) alternates sign
- Space (0) remains zero

MATLAB Code

```
clear;
clc;

% Input binary sequence
b = [0 1 0 0 1 0];

% Number of bits
n = length(b);

% Time vector
t = 0:0.001:n;

% Convert binary data to waveform
for i = 1:n
    if (b(i) == 0)
        b_p(i) = 0;      % Binary 0 remains 0
    else
        b_p(i) = 1;      % Binary 1 becomes 1
```

```

        end
    end

    % Generate rectangular waveform
    for j = 1:n
        bw(j*1000:(j+1)*1000) = b_p(j);
    end

    bw = bw(1000:end);

    % Plot input signal
    subplot(2,1,1)
    plot(t,bw)
    grid on
    title('Input Binary Signal')
    xlabel('Time')
    ylabel('Amplitude')

    % Generate Bipolar AMI signal
    lastbit = 1;

    for i = 1:n
        if b(i) == 1
            x(i*1000:(i+1)*1000) = -lastbit;
            lastbit = -lastbit;
        else
            x(i*1000:(i+1)*1000) = 0;
        end
    end

    x = x(1000:end);

    % Plot AMI encoded signal
    subplot(2,1,2)
    plot(t,x)
    grid on
    title('Bipolar AMI Encoded Signal')
    xlabel('Time')
    ylabel('Amplitude')

```

Code Explanation

1. Initialization

```

clc;
clear;
close all;

```

- `clc` → Clears command window

- `clear` → Removes all variables

2. Define Binary Input

```
b = [1 1 0 0 1 0];
n = length(b);
```

- `b` contains the binary input sequence
- `n` gives the total number of bits

3. Create Time Vector

```
t = 0:0.001:n;
```

- Creates the time axis from 0 to `n`
- Step size 0.001 gives a smooth plot

4. Convert Binary Data

```
if (b(i) == 0)
    b_p(i) = 0;
else
    b_p(i) = 1;
```

Converts:

- $0 \rightarrow 0$
- $1 \rightarrow 1$

Used for generating input waveform.

5. Generate Binary Waveform

```
bw(j*1000:(j+1)*1000) = b_p(j);
```

- Each bit is expanded over 1000 samples
- Creates rectangular waveform

6. Initialize Alternating Bit

```
lastbit = 1;
```

This stores previous polarity.

Initially:

```
lastbit=+1
```

7. Generate Bipolar AMI Signal

```
if b(i) == 1
    x(i*1000:(i+1)*1000) = -lastbit;
    lastbit = -lastbit;
else
    x(i*1000:(i+1)*1000) = 0;
end
```

This is the main AMI logic.

For binary 1: polarity alternates: $+1, -1, +1, -1$

For binary 0: 0

8. Plotting

Subplot 1 → Bipolar input signal

Subplot 2 → Bipolar AMI encoded signal

Result:

The MATLAB program produces:

1. Bipolar digital signal

2. Bipolar AMI encoded waveform

Observations:

- For binary 0 $\rightarrow$ 0 voltage
- Binary 1 $\rightarrow$ alternating +1/-1
- Successive 1s invert polarity

Conclusion: The experiment successfully demonstrated Bipolar AMI Encoding using MATLAB. Binary 0 was represented by zero voltage, while binary 1 was represented by alternating positive and negative voltage levels. The output waveform verified the Alternate Mark Inversion principle.

Assignment: Write a MATLAB program to generate the Pseudoternary encoded signal for a binary sequence given as input by the user.